

InsurMinds

Plataforma inteligente para análise e comparação de apólices D&O

Relatório técnico

Protótipo acadêmico que recebe duas apólices em PDF ou imagem, extrai campos D&O com apoio de um LLM, compara valores e apresenta evidências por página e trecho.

Estado	Escopo implementado	Validação pendente
Protótipo integrado em código; execução de ponta a ponta com chave real ainda não confirmada.	Ingestão/OCR, schema Pydantic, adaptador OpenAI, comparação, interface e histórico SQLite opcional.	Teste com modelo real, golden dataset, validação da especialista e gravação final da demonstração.

Equipa

Jheniffer Guimarães: IA, agentes e comparação. Os nomes e responsabilidades dos restantes integrantes devem ser completados pelo grupo.

Documento preparado em 3 de outubro de 2026. O protótipo não avalia qual apólice é melhor e não substitui análise de seguros ou jurídica.

1. Requisitos do projeto

O enunciado pede um MVP demonstrável que leia documentos, extraia e organize informações, compare pelo menos duas apólices, use IA generativa e apresente resultados numa interface. O quadro reflete o código presente no workspace nesta data.

Requisito	Estado	Evidência / lacuna
Ler PDF ou imagem	Implementado na ingestão	PDF nativo e OCR de PDF digitalizado, PNG e JPEG em src/ingestion/
Extrair campos D&O; automaticamente	Implementado, requer credencial	ExtractionAgent chama OpenAI e valida retorno; falta uma execução real nesta revisão.
Estruturar os dados	Implementado	PolicyExtraction usa Pydantic, estados de incerteza e citações verificadas.
Comparar duas apólices	Implementado em código	ComparisonEngine compara LMG, franquias, limites, coberturas e exclusões.
Apresentar diferenças	Implementado na interface	Streamlit apresenta tabela, citações, JSON e exportação CSV.
Usar modelo generativo	Integrado, não executado	OpenAIChatClient usa Chat Completions; depende de chave/modelo e autorização para envio.
Interface demonstrável	Implementada, falta validação live	app/main.py recebe dois documentos; Streamlit não estava instalado no runtime de verificação.
Armazenar/consultar resultados	Opcional local	SQLiteAnalysisRepository permite histórico local, desligado por padrão.
Relatório, deck e vídeo	Parcial	Relatório técnico, pitch deck e roteiro foram preparados; o vídeo final depende de uma demonstração live.

Conclusão de conformidade

O código agora cobre o fluxo funcional pretendido, mas a entrega ainda não pode ser declarada validada: é necessário executar a interface com uma chave/modelo habilitado, conferir duas apólices autorizadas e concluir os artefactos e dados da equipa.

2. Arquitetura e fluxo

A aplicação separa ingestão, extração, comparação, explicação e persistência. O texto e as referências de origem fluem entre componentes; os PDFs originais não são guardados no histórico SQLite.

Etapa	Componente	Responsabilidade
Receção e OCR	DocumentIngestionPipeline	Classifica PDF/PNG/JPEG, extrai texto nativo ou OCR, limpa e divide em chunks por página.
Extração	ExtractionAgent + OpenAIChatClient	Envia lotes de chunks ao modelo, pede JSON e valida campos com Pydantic.
Normalização	ClauseNormalizer	Aplica códigos apenas a sinónimos explícitos conhecidos; mantém rótulos não reconhecidos.
Comparação	ComparisonEngine	Aplica regras determinísticas a valores normalizados e produz ComparisonReport.
Explicação	ExplanationAgent	Opcionalmente descreve diferenças com base apenas no relatório estruturado.
Interface e consulta	Streamlit + SQLite opcional	Carrega duas apólices, exhibe resultados/evidências, exporta CSV e consulta análises guardadas.

Proveniência

Cada evidência contém identificador do documento, número da página, chunk e citação literal. O agente confirma que a citação ocorre no chunk indicado antes de aceitar o JSON. Valores não localizados, ambíguos e não aplicáveis têm estados explícitos.

Tratamento de documentos longos

O agente envia até oito chunks por chamada para reduzir risco de exceder o contexto. A aplicação pode fazer várias chamadas por apólice, aumentando custo e tempo. Conflitos entre lotes ficam ambíguos para revisão.

3. Dados e comparação

O schema cobre identificação da apólice, seguradora, segurado, vigência, retroatividade, períodos complementares/suplementares, run-off, LMG, LMI/sublimites, franquias, coberturas, exclusões e extensões. Cada campo usa estado, valor e evidências.

Regra	Comportamento
Valor extraído	Aceite apenas se validado pelo schema e acompanhado por evidência no texto de origem.
Valor não localizado	Representado como not_found; não é convertido em cobertura inexistente.
Conflito entre trechos	Marcado como ambiguous e encaminhado à revisão humana.
Valores monetários	Comparados apenas quando as moedas coincidem; diferença nominal não implica adequação.
Franquia percentual	Comparável apenas com base de cálculo igual e informada; tipo misto fica para revisão.
Cobertura/exclusão não localizada no par	Apresentada como pendência de revisão, não como ausência confirmada.
Cláusula jurídica divergente	Exibe os trechos encontrados e solicita interpretação especializada.

Exemplo ilustrativo do golden dataset fornecido

Critério	Apólice A (exemplo Chubb)	Apólice B (exemplo AIG)	Diferença esperada
LMG	R\$ 50.000.000	R\$ 100.000.000	B tem valor nominal maior
Franquia	R\$ 50.000	R\$ 100.000	B tem valor nominal maior
Fraude e guerra	Mencionadas	Mencionadas	Comparar redação e gatilhos, não apenas o rótulo

Os valores acima vêm do documento de regras de negócio e não representam resultado processado pelo app.

4. Implementação e decisões

Decisão	Motivo
Pydantic para contrato	Rejeita JSON fora do formato, restringe estados e valida datas/campos.
Comparação determinística	Mantém cálculos e direção das diferenças reprodutíveis; deixa interpretação jurídica para pessoa especialista.
Adaptador OpenAI desacoplado	Usa interface simples para JSON/texto, sem SDK de fornecedor adicional.
Confirmação antes de envio externo	A pessoa confirma que pode enviar o texto à API antes de cada análise.
SQLite opcional e local	Permite guardar e consultar resultados sem armazenar os PDFs; salvamento vem desligado por padrão.
CSV e evidências na interface	Facilitam revisão por corretor, gestor de risco ou jurídico.

Execução

Instalar dependências com `python -m pip install -r requirements.txt`, instalar Tesseract com idioma português, configurar `OPENAI_API_KEY` e `OPENAI_MODEL` e iniciar com `python -m streamlit run app/main.py`. O README descreve o fluxo e o uso do histórico SQLite.

Validação realizada neste workspace

A compilação de `app/src/tests` passou. Seis verificações focadas de extração por lotes, verificação de citações, comparação, comportamento conservador, SQLite e formato JSON do adaptador passaram usando cliente simulado. Não houve chamada paga à API. A suíte `pytest` não foi executada porque `pytest` e `Streamlit` não estão instalados no runtime disponível; a interface ainda precisa de demonstração live.

Privacidade

O adaptador envia texto extraído e prompts à API OpenAI. O app avisa a pessoa e exige confirmação. A autorização da equipa para cada documento permanece necessária. Resultados SQLite contêm valores e excertos derivados da apólice e devem ser protegidos no dispositivo.

5. Limitações e evolução

Pendente	Próximo passo
Execução real do LLM	Configurar modelo habilitado e validar campos/evidências com apólices autorizadas.
Golden dataset	Anotar documentos sintéticos, comparar extrações e medir precisão, recall e F1 por campo.
Taxonomia inicial	Validar aliases para Side A/B/C, EPL, investigações, exclusões e extensões com especialista.
Coberturas e gatilhos	Desenvolver comparação semântica com casos de revisão humana e sem inferir equivalência jurídica.
Grupo e relatório final	Completar nomes de todos os integrantes, rever relatório, pitch e vídeo de até cinco minutos.
Integração e operação	Executar instalação limpa, app Streamlit e roteiro de demonstração antes de publicar.

Fontes consultadas

1. Enunciado “Projeto Final - Plataforma Inteligente para Análise e Comparação de Apólices D&O”, fornecido pela equipa, pp. 20-23. Define requisitos, entregáveis, critérios e prazo de 6 de outubro de 2026.
2. Regras de Negócio do Projeto Final 6 InsurMinds I2A2, fornecidas em data/do/referencias/. Fonte do modelo de dados, campos, golden dataset ilustrativo e matriz de coberturas.
3. Repositório InsurMinds, branch main no commit 38ba589 (pipeline de ingestão/OCR), mais alterações locais descritas em README.md e docs/arquitetura.md. Os módulos desta versão local ainda não foram commitados.
4. OpenAI API Reference, Chat Completions: <https://platform.openai.com/docs/api-reference/chat/create>. Consultada para o formato da chamada ao serviço.

A conclusão académica depende de uma execução demonstrável com duas apólices autorizadas, da revisão técnica/domínio e da entrega dos artefactos finais.